Object-Oriented Programming

Core Concepts, Architecture & Implementation

A Comprehensive 5-Page Technical Overview

Software Engineering Architecture Series

01. Paradigm Foundations: Paradigm shift & Classes

Object-Oriented Programming (OOP) is a programming paradigm organized around **data**, or objects, rather than functions and logic. It models real-world entities to make code modular, reusable, and scalable.

The Class (The Blueprint)

A blueprint or template that defines the structure and behavior (attributes and methods) that objects of the class will possess.

```
class Car { String brand; void drive()
{ // logic } }
```

The Object (The Instance)

A self-contained execution instance of a class, occupying memory space and holding actual state values representing a concrete entity.

```
Car myCar = new Car(); myCar.brand =
"Tesla"; myCar.drive();
```

02. Data Integrity & Hierarchies

Encapsulation (Data Hiding)

The practice of bundling data and methods inside a single unit, restricting direct access to shield the state from accidental modification.

- **Mechanisms:** Achieved using access modifiers (`private`, `protected`, `public`).
- **Accessors:** Controlled updates via Getter and Setter methods.

```
private double balance; public void deposit(double amt) { if(amt > 0) balance += amt; }
```

Inheritance (Code Reuse)

The mechanism by which one class derives state and behavior from a parent class, establishing an **"Is-A" relationship** hierarchy.

- **Parent (Superclass):** Base definition provider.
- **Child (Subclass):** Inherits, extends, or alters parent attributes and capabilities.

```
class ElectricCar extends Car { int batteryCapacity; // Inherits brand and drive() }
```

03. Polymorphism & Complexity Reduction

Polymorphism (Many Forms)

Allows a single interface or entity to trigger different behaviors depending on context or underlying type.

- **Compile-time (Overloading):** Same method name, different parameters within the same class.
- **Runtime (Overriding):** Child class redefines a parent method dynamically.

```
// Runtime dynamic dispatch Animal a =
new Dog(); a.makeSound(); // Outputs:
"Bark"
```

Abstraction (Interface vs Implementation)

Hides complex internal implementation details and exposes only essential operations to the user.

- **Abstract Classes:** Template classes that cannot be directly instantiated, may contain abstract methods.
- **Interfaces:** Pure blueprints specifying *what* a class must do, but not *how*.

```
interface DBConnection { void
connect(); // structural signature }
```

04. Engineering Benefits & Design Paradigms

Utilizing OOP properly enhances long-term software design stability, system scaling, and maintenance routines.

Primary System Benefits

Modularity: Distinct objects decouple structural bugs, allowing localized troubleshooting.

Maintainability: Inheritance simplifies code expansions without modifying existing foundational architecture.

Security: Encapsulation guarantees data properties remain valid and non-corruptible by external routines.

The SOLID Framework Overview

Modern OOP development follows key architecture practices:

Single Responsibility	Class should have one reason to change.
Open/Closed	Open for extension, closed for modification.
Liskov Substitution	Derived classes must replace base classes cleanly.
Interface Segregation	Avoid forcing dependency on unused interfaces.
Dependency Inversion	Depend on abstractions, not concrete specifics.